M68KWB Documentation

Latest edit: Feb 23rd, 2026

- WHAT IS IT?
  - Features
  - Production cycle
  - UI Quick Overview
  - Advanced debugging features?
    - Accessing registers
    - Logging
    - The registers viewer
    - Breakpoints
    - Error context
    - Memory
    - The Toolbox
    - Debug Primitives
    - Paranoid mode
    - Locks
    - Watchdog
    - MISC
  - Command Line
  - Copperlist (OCS only)
  - Emulation matrix
    - AMIGA
    - ATARI ST/STe
- WHY SHOULD YOU USE THIS?
  - If you're an Amiga Coder
  - If you're an Atari Coder
- THIS IS A WEB TOOL
  - You’re not a web expert? Not a problem
  - About File Paths
  - Tools needed to code and test
- BROWSING THE FX LIST
- CREATING YOUR OWN FX
- USING THE DEBUGGER
- THE JAVASCRIPT / ASM PING-PONG
- CROSS PLATFORM DEVELOPMENT
- LIMITATIONS & ANNOYING STUFF I NEED TO FIX

Assembler
Code and data sections
CPU Limitations
When shit hits the fan
THE AMIGA DISPLAY

WHAT IS IT?

M68KWB is an Amiga and Atari ST/STe development tool that allows you to smoothly go from javascript prototyping to asm implementation. It's also useful if you only need to develop in pure assembly from scratch. I've been developing and using it for years now, and it really allowed me to code much faster, so I guess it's time to share now. If you need help / have questions:
email: soundydeadliners@gmail.com

Discord: soundy4264

Features

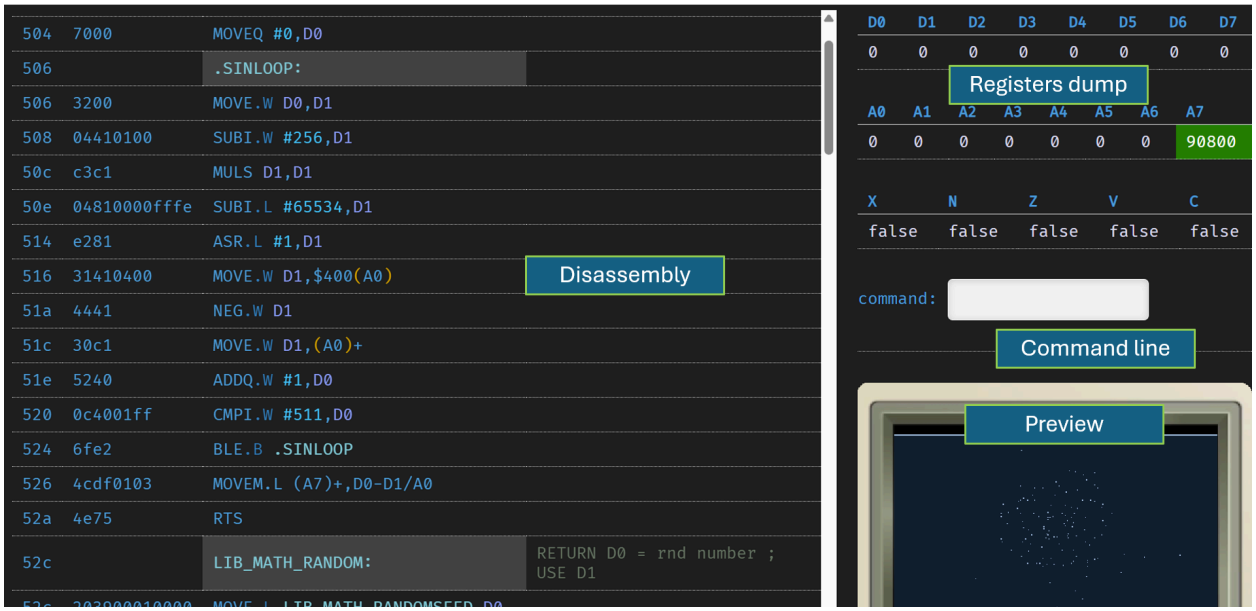
- **An assembler**, to assemble the source code you typed to actual opcodes in memory. The assembled instructions remain in RAM, you can't save them to disk or produce an executable for your final platform with it, so to produce your final demo's exe, you'll still need vasm (or any similar tool) in the end.
- **A disassembler**, because you may have used auto-modified code so the debugger must show you the proper instructions.
- **A debugger**, with classic and advanced features to trace your code, trigger breakpoints, etc.
- **Rudimentary emulation** of target platforms (Amiga OCS, Atari ST/STe for the moment), providing bitplane emulation, and a few extra hardware features. Note this tool is meant to make your m68k coding easier only. It does not emulate CPU cycles, DMA cycles, nor is it a full emulator for the Amiga or the ST.

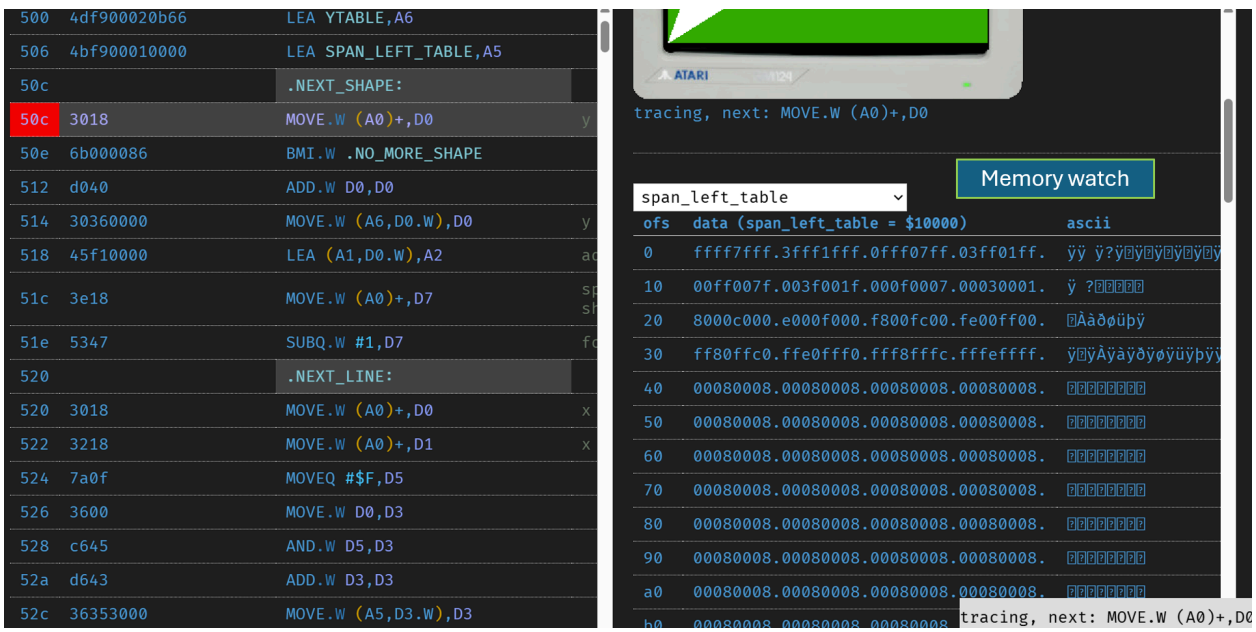
Production cycle

1. **Write your code**. You need a code editor for this. I highly recommend Visual Studio Code, and any m68k syntax extension so that you have syntax coloring when typing your code.
2. **Test & Debug**. Open M68KWB on your favorite browser, see how it works, debug, etc.
3. **Test & Debug**. As explained above, M68KWB does not fully emulate your target machine. Use your favorite assembler (vasm, Devpac, …) to generate an executable and test it with your favorite emulator (WinUAE, Hatari, …). If you're targeting Amiga, you can use Bartman's VS Code extension here too.

When prototyping your FX, you will spend a lot of time iterating between steps 1 & 2. Obviously, you don't need to check on the actual emulator as long as your FX does not work ;) Also, you may come back from step 3 to step 1

UI Quick Overview





Advanced debugging features?

Memory protection, code breakpoints, hardware breakpoints, value breakpoints, math overflow detection, but the most important is the ability to write and execute javascript in the middle of your asm code (called ***inline JS***). For example, imagine you have a table offset in `d0`, and you want to make sure you don't fetch outside of your table:

```
add.w d0,d0
;>JS if (d0.w > 1023) breakpoint("texture index overflow: " + d0.w);
;>JS if (d0.w < 0) breakpoint("texture index underflow: " + d0.w);
move.w (a0,d0.w),d0
```

This is a very simple example, but much more complex things can be achieved. Just note from the above:

- Any line starting with "`;>JS`" is considered as ***inline* JS**. If you assemble this with vasm, devpac or asmOne for example, it will be considered as a comment as it starts with '`;`'. So you can have as many as you want, it won't affect your final product (a bit like an assert in C).
- There's plenty of helpers to access asm labels, registers, and memory from inline JS (such as the `d0.w` above). Debugging primitives (draw points, lines, etc.) are also available. There's no limit to what you can do, you may call very complex JS functions featuring thousands of lines right in between 2 asm lines.

Here's a list of the available helpers from *inline* JS:

Accessing registers

Registers are stored by M68KWB in 2 Javascript `Uint32Array(8)`: `regs.a`, and `regs.d`. For example, the 2 below instructions do exactly the same thing:

```
;>JS regs.a[2] = 0;
move.l #0,a2      ; same as above
```

```
;>JS regs.d[4] = 42;
move.l #42,d4     ; same as above
```

Using regs.a[] and regs.d[] is currently the only way to write registers from *inline JS*. Now, it can be cumbersome to use this to **read** values and log them. For example, if you want to log the value of the lower 16 bits of d0, you need to write:

```
;>JS print(regs.d[0] & 0xffff)
```

Actually, there's a series of helpers to read (and only read) registers values more easily from *inline* JS:

`<type><index>.<signed><part>`

type:	`d or a`
signed:	`i` (for signed), `u` (for unsigned). Defaults to signed if omitted
part:	`b, w, l or h`

examples:

`d0.ub`	unsigned 8 lower bits of d0
`d5.il`	signed 32 bits of d5
`d4.uw`	unsigned 16 lower bits of d4
`a2.w`	signed lower 16 bits of a2
`d1.ih`	signed higher 16 bits of d1

Logging

- Print a line in your browser's debugger (similar to console.log):

  ```
  ;>JS print("bad value d0 = " + d0.w);
  ```

- Show a "debug log" floating window at the top left of the screen with some debug info

  ```
  ;>JS ShowDebugLog("init done");
  ```

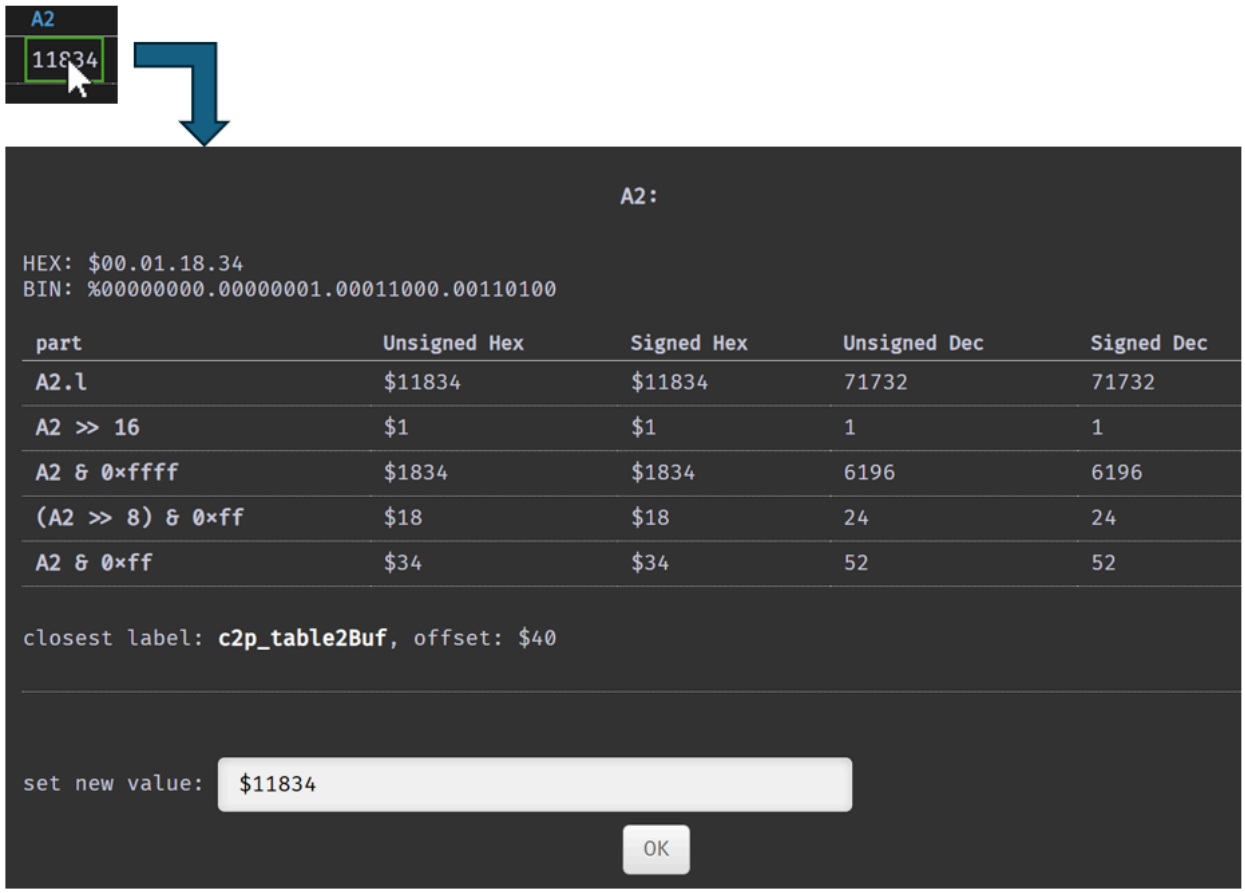
  ```
  ; >JS ShowDebugLog("a0: " + a0.l);
  ```

  ```
  ; >JS ShowDebugLog("offset 1:" + d0.w + "<br>offset 2: " + d1.w); //
  classic HTML tags work (here <br>) in the debug log window.
  ```

  ```
  ; >JS HideDebugLog(); // hide the debug log window
  ```

The registers viewer

At the top-right of the screen, click on the value of any data or address register to open this register's detailed window:



From there, you can check the value of the different parts of the register (signed, unsigned, hex, and decimal), and you can also assign a new value to the register (using "set new value").

Note that for address registers only, you also have the "closest label / offset" information. M68KWB starts from the current address stored in the register, and finds the closest known label below that address.

Breakpoints

- Unconditional Breakpoint: `debug(message)`

```
;>JS if (d0.uw > 50000) debug("overflow"); // trigger a breakpoint
;>JS if (d0.uw > 50000) breakpoint("overflow"); // same as above
⇒ The message is optional and can be left blank
```

- Value Breakpoint: `DEBUGGER_ValueBreakpoint(adrs, value, size)`

```
;>JS DEBUGGER_ValueBreakpoint(label("myPointer"), 0xdeadbeef, 4);
```

⇒ Only 1 Value Breakpoint is active at a time. If you declare one, you overwrite the previous one.

⇒ Note the use of Javascript number syntax (hence the "0x" instead of "$")

- Hardware Breakpoint: `DEBUGGER_HWBreakpoint(index, adrs, size)`

```
;>JS DEBUGGER_HWBreakpoint(0, label("myPointer"), 4);
```

⇒ Same as WinUAE's "w" command: index, adrs, size

⇒ "index" can be 0, 1, 2 or 3 and allows you to choose which of the 4 available HW breakpoints shall be used. You can change the number of available HW breakpoints using `DEBUGGER_CONFIG.MAX_HW_BPT` in config.js, though, knowing that setting it too high can impact performance.

- You can also access a "Address Breakpoints" window at the bottom of the Toolbox (shift + t).

- You can also have an inline JS be executed automatically after every CPU instruction. For example, if you have no clue where your d1 register is cleared while it shouldn't, you can use:

  ```
  ;>JS CPU_DBG_AFTERINSTR =  "if (d1.iw == 0) debug()";
  ```

  …. some code you suspect is wrong….

  ```
  ;>JS CPU_DBG_AFTERINSTR =  null;
  ```

  Notes:

  - The code to execute is Javascript, and must be a string (hence the quotes).
  - This can slow down everything significantly as the code between quotes will be executed after every single CPU instruction.
  - Set variable `CPU_DBG_AFTERINSTR` back to `null` to stop tests.

Error context

You can add extra debug info along the way that will be shown only if an error raises. consider the below code (found in Tutorial 7: “`tut7_errorContext.asm`“)

```
      ;>JS DebugCtx_reset("update loop");
      ;>JS DBGVAR.yiter = 0;
      move.w  #1,d7
.yloop:
      ;>JS DBGVAR.yiter++;
      ;>JS DebugCtx_enter("y iter: " + DBGVAR.yiter);
      move.w  #1,d6
      ;>JS DBGVAR.xiter = 0;
.xloop:
      ;>JS DBGVAR.xiter++;
      ;>JS DebugCtx_enter("x iter: " + DBGVAR.xiter);
      move.w  d7,d0
      add.w   d0,d0
      add.w   d6,d0
      ;>JS DebugCtx_log("value: " + d0.iw);
      ;>JS if (d0.iw == 1) debug("wrong value for x=" + d6.iw + ", y=" +
d7.iw);
      ;>JS DebugCtx_exit();
      dbra    d6,.xloop
      ;>JS DebugCtx_exit();
      dbra    d7,.yloop
      ;>JS DebugCtx_reset(); // clear context for later messages
```

Here’s the error message you will see when D0 equals 1 (all items are shown in a collapsible tree):

```
file: FX/ATARI/MAPPING/R24_SPAN_NOC2P.ASM
line: 40
wrong value for x=1, y=0
▼Context
  update loop
  ▼y iter: 1
    ▼x iter: 1
      value: 3
    ▼x iter: 2
      value: 2
  ▼y iter: 2
    ▼x iter: 1
      value: 1
```

Here's a detailed description of what is done:

1. Call `DebugCtx_reset` to erase any previous context provided by another previous routine.
2. Init a persistent JS variable (using `DBGVAR`) to keep track of the current iteration index (could use d7 obviously, this is for the sake of the example). This variable is updated in the y-loop.
3. For each y iteration, log we are entering / exiting this iteration using `DebugCtx_enter` / `DebugCtx_exit`. This is totally optional, the only effect this has is that logged messages are indented accordingly (the arrows you see in the above screenshot). Do the same for x-loop.
4. Use `DebugCtx_log` to log a message.

Whenever a breakpoint of an error is reached, the context will be added to the error message. Remember that `DebugCtx_enter` / `DebugCtx_exit` are totally optional. You may just stack `DebugCtx_log` with no hierarchy,

Memory

Memory is a huge Javascript `Uint8Array` called `MACHINE.ram[].` You may access this array from JS, but helpers are here for you:

Reading from a given address: getram(<adrs>, <bytes>(1,2 or 4), <signed> (true/false)

`;>JS getram(a0.ul, 2, true)` ⇒ returns the signed word at (a0)

Writing to a given address: setram(<value>,<adrs>, <bytes>(1,2 or 4))

`;>JS setram(42,a0.ul, 2)` ⇒ same as move.w #42,(a0)

Fetching labels: label(<name>)

```
BUFSIZE     EQU 1024
```

```
;>JS if(a5.ul-label("startBuffer") >= const("BUFSIZE")) debug("overflow");
```

Yep, const(<name>) fetches the value of a constant declared in asm using "`=`" or "`EQU`".

Fetching variables: variable(<label>, <size>, <signed>)

```
;>JS print("frame: " + variable("curFrame",2,false));
```

Now, imagine you need to fetch a variable within a struct:

```
      rsreset
var1        rs.w  1
var2        rs.w  1
structLen   rs.b  0
…
myStruct:   ds.b  structLen
```

```
;>JS if(struct("myStruct", "var2", 2, false) > 12) debug("overflow");
```

The syntax being: struct("<label name>", "<variable name>", size (1,2 or4), signed (true/false))

Protecting memory:

```
; trigger a breakpoint if anyone fucks with my buffer

;>JS CPU_DBG_WRITE_FORBID_START = label("myBuffer");

;>JS CPU_DBG_WRITE_FORBID_END = label("myBuffer") + const("bufferSize");
```

Another way to do the same thing:

```
;>JS ProtectBuffer(label("myBuffer"), const("bufferSize")) ; <adrs>,<size>

…

; we can update the buffer again now

;>JS CPU_DBG_WRITE_FORBID_START = -1;

;>JS CPU_DBG_WRITE_FORBID_END = -1;



; I'm supposed to write only to the bitplanes in this routine

;>JS CPU_DBG_WRITE_ALLOW_START = label("buf");

;>JS CPU_DBG_WRITE_ALLOW_END = CPU_DBG_WRITE_ALLOW_START + 32000;

…

;>JS CPU_DBG_WRITE_ALLOW_START = 0;

;>JS CPU_DBG_WRITE_ALLOW_END = 999999;

rts
```

<u>Another syntax for the same thing</u>:

limitWrite(<min adrs>, <max adrs>

limitRead(<min adrs>, <max adrs>

```
;>JS limitWrite(a0.ul, a0.ul+SCR_W_BYTES*SCR_H)
```

⇒ limits memory writes to a zone between the current value of `A0` and `A0+SCR_W_BYTES*SCR_H`. Basically makes sure that we won't write beyond the bitplane pointed by `A0`.

```
;>JS limitRead(label("fontData"),
label("fontData")+TOOLS.getFileSizeFromURL("data/font8x8_bitplanes.bin"))
```

⇒ limits memory reads to a zone between label `fontData` and `fontData`+the size of my font file.
`;>JS freeRead()` ⇒ cancels any previous `limitRead`

`;>JS freeWrite()`⇒ cancels any previous `limitWrite`

Watching Memory:

There's a memory watch at the bottom of the right pane. The UI allows you to set manually which address you want to watch, but it can be done in the code too, so that you can points the debugger to the right address automatically, which saves lots of time when debugging:

```
; >JS DBG_setDumpReg('a0');  ==> watch memory pointed by register a0

; >JS DBG_setDumpReg('myBuffer');  ==> watch memory pointed by label myBuffer
```

See also "**The Toolbox**" chapter below for more advanced ways to watch memory.

The Toolbox

Press `shift+t` to toggle it on/off. The Toolbox is a draggable window featuring several tools, one of them being the **watch**. If you tick the watch checkbox in the toolbox, any watches declared in your code will be shown. Here's how to declare a watch using *inline* JS:

```
;>JS WATCHES.add(<name>, <address>, <type>, <base>);
```

- `name` : the label that will appear in the watch
- `address` : the address to be watched
- `type` : `WATCH_BYTE`, `WATCH_WORD` or `WATCH_LONG`
- `base` : `WATCH_BIN, WATCH_DEC or WATCH_HEX`

Examples:

- Show/edit the value of a frame counter in hexadecimal:

```
;>JS WATCHES.add("frame", label("frame"), WATCH_WORD, WATCH_HEX);

update:

      add.w #1,frame

frame:

      dc.w  0
```

- Show/edit the value of a "hardcoded" address in memory:

```
;>JS WATCHES.add("color0", 0xdff180, WATCH_WORD, WATCH_HEX); // Amiga

;>JS WATCHES.add("color0", 0xff8240, WATCH_WORD, WATCH_HEX); // Atari
```

Note that the address is a Javascript number (hence the "0x" instead of "$" for hex number). This also means that arithmetic operations are ok, e.g. to watch color8, you could type:

```
;>JS WATCHES.add("color8", 0xdff180+8*2, WATCH_WORD, WATCH_HEX); // Amiga

;>JS WATCHES.add("color8", 0xff18240+8*2, WATCH_WORD, WATCH_HEX); // Atari
```

Debug Primitives

```
;>JS DEBUGPRIM.addLine(d0.h, d0.w, d1.h, d1.w, 0xff00ff); // x1,y1,x2,y2, HTML
color

;>JS DEBUGPRIM.addPoint(d0.w, d1.w, 0xff00ff, 4); // x,y,HTML color,radius
```

Paranoid mode

This mode is actually the default mode ;) It constantly checks for reading/writing to wrong addresses, jumping outside of the code section, data breakpoints, etc. If you want to get funky with your code (self-modified code, code generated in data section, etc.), you can disable and re-enable paranoid mode your asm file as such:

```
;>JS DEBUGGER_paranoid = false
```

... do your nasty stuff here

```
;>JS DEBUGGER_paranoid = true
```

It's recommended to keep `DEBUGGER_paranoid` to `true` as much as possible to guarantee the debugger is behaving properly.

Note also that verifications can be disabled on a per line basis just by adding `M68KWB_NOERROR` anywhere in a the comment following the instruction (on the same line)

Locks

Sometimes you want to make sure you don't overwrite a register to store something while you are still using it for something else. here's what you need:

```
; >JS lock(<register>,<mask>,<reason>);
```

- **register**: name of the register (d0..d7/a0..a7), case insensitive.
- **mask**: bits to protect (out of the register's 32 bits). Expressed as a JS number. Defaults to 0xffffffff if not given.
- **reason**: optional string (between quotes) to give an extra content to the error message

e.g.:

```
; >JS lock('d0', 0xffff, 'texture table offset'); // make sure the low 16 bits of d0 are not modified by any instruction.
```

After every CPU instruction, M68KWB will check if the protected bits were modified and trigger a breakpoint if it's the case. You will unlock a register as follows:

```
; >JS unlock('d0');
```

Note you can't unlock a mask, the full register is unlocked (but you may relock another mask immediately).

Now, you sometimes want to update the value of a locked register without unlocking it, e.g. keep it locked but change the locked value, here's the procedure:

```
; >JS lock('d0', 0xffff, 'texture table offset');

…

move.w #12,d0     ; M68KWB_NOERROR I know what I'm doing…
```

Just add `M68KWB_NOERROR` anywhere in a the comment following the instruction (on the same line)

Watchdog

M68KWB counts how many instructions have been executed since the last CPU invoke (e.g. at each update) and triggers a breakpoint if a maximum is reached. This makes infinite loops easier to investigate. There are 3 ways to modify this max count:

- Change the value of `watchdog_maxInstr` in `CPU_CONFIG` in file `config.js`. Will change the setting globally for all fx (bad).
- `In file user_fx.js, for your fx, add a field "watchdog:"` with the new value. This will change the value for your whole fx (meh, but quick),
- Change it using *inline asm* (better!) to disable the watchdog only for the portion of code that needs it:

```
;>JS CPU_CONFIG.watchdog_maxInstr = 0; // disable watchdog because I'm about to launch a very long precalc because I'm a cheater that stores a big table in RAM

… do my nasty precalc here

;>JS CPU_CONFIG.watchdog_maxInstr = 400000; // re-enable watchdog
```

MISC

- **Pausing / Changing execution speed**

`;>JS onNewRunningSpeed(<amount>);` changes the execution speed (0..100), so that you can slow down for parts you want to debug, and have more time to see what's happening. 100 is normal speed

```
;>JS onNewRunningSpeed(30); ==> run at 30% of the normal speed (70% slow down)

....

;>JS onNewRunningSpeed(100); ==> back to full speed
```

`;>JS SET_PAUSE(true)` : just call this at the end of your update, you'll have to press the `'p'` key to unpause.

- ### Multiple-line inline JS

You can't declare a JS variable on a line and use it further. you have to set it as a field of DBGVAR.

<u>The below will NOT work</u>:

```
;>JS let oldD0 = d0.l // save d0 in a temp local variable
…                     // mess with d0
;>JS d0.l = oldD0     // nope, oldD0 will be undefined
```

<u>Do this instead</u>:

```
;>JS DBGVAR.oldD0 = d0.l      // save d0 as a field of a global lvariable
…                             // mess with d0
;>JS d0.l = DBGVAR.oldD0      // yep, that works
```

Check `fx/amiga/Sphere/v4/sphv4.asm`, and look for "`DBGVAR.`", you'll see tons of usages.

How the fuck did I end up here?

Yeah, sometimes I break on a line and wonder what the fuck is happening. 2 tools for that:

1. **Where am I?**

   just press the ‘w’ key while tracing

Where am I?

```
Just executed: ADD.W D0,D1
file: fx/amiga/StarField/v2/strfldv2.asm, line 54
```

Values on Stack

instr	file/line
BSR PLOT	file: fx/amiga/StarField/v2/strfldv2.asm, line 144
drawStars	Invoked from JS

Last taken branches

ofs	instr	file/line
678	BSR PLOT	file: fx/amiga/StarField/v2/strfldv2.asm, line 144
5da	RTS	file: fx/amiga/StarField/v2/strfldv2.asm, line 76
66e	BSR PROJECT	file: fx/amiga/StarField/v2/strfldv2.asm, line 142
666	BLE.B .ZOK	file: fx/amiga/StarField/v2/strfldv2.asm, line 138
608	RTS	file: fx/amiga/StarField/v2/strfldv2.asm, line 99
654	BSR ROT	file: fx/amiga/StarField/v2/strfldv2.asm, line 129

OK

2. **Time Machine a.k.a CPU undo/redo**

When reaching a breakpoint, you may trace back previous instructions. M68KWB stores the registers values and IP for the last 256 instructions (reverts writes to RAM also, except blitter writes). Use keys x&y to play with the time machine.

Command Line

Debugging commands can be typed in the "command" text field. Supported so far:

- Hardware breakpoint: same as WinUAE: `w <index> <address> <size>`

  * `index` : 0,1,2 or 3 (only 4 hardware breakpoints supported, look for `MAX_HW_BPT` in `config.js` to change this).

* `address:` hex address of the HW breakpoint (do not prefix with '$', just type the hex address)

* `size:` hex size of the HW breakpoint (do not prefix with '$', just type the hex size)

for example, to break on the long word at address 100: `w 0 100 4`

- Value breakpoint: `v <address> <value> <size>`

* `address:` hex address of the Value breakpoint (do not prefix with '$', just type the hex address)

* `size:` 1, 2 or 4

for example, to break when the word at $100 equals $1234: `v 100 1234 2`

Note that only 1 value breakpoint is supported at a time

- Set memory dump address: `m <address>`

* `address: hex address to dump (do not prefix with '$', just type the hex address)`

- Change register value : just type what you need. examples:

* `a0 = 0 ==> set a0 to 0`

* `a0 = d0 ==> copies d0's value to a0`

* `a0 = d2*5 ==> copies 5 x d2's value to a0`

* `d0 = $ff ==> set d0 to 255`

* `d0 = %111 ==> set d0 to 7`

- `Change current instruction: j <offset>`

* `offset:` ofs of the dest line as it appears in the debugger's left column

Note: `ofs` can be considered as the address of an instruction, it’s the same thing.

Copperlist (OCS only)

Press shift+c

adrs	opcode	instruction
1279c	008e,4c81	DIWSTRT = $4c81
127a0	0090,00c1	DIWSTOP = $c1
127a4	0092,0038	DDFSTRT = $38
127a8	0094,00d0	DDFSTOP = $d0
127ac	00e0,0001	BPL1PTH = $1
127b0	00e2,0b7c	BPL1PTL = $b7c
127b4	0120,0000	SPR0PTH = $0
127b8	0122,0000	SPR0PTL = $0
127bc	0124,0000	SPR1PTH = $0
127c0	0126,0000	SPR1PTL = $0
127c4	0128,0000	SPR2PTH = $0
127c8	012a,0000	SPR2PTL = $0
127cc	012c,0000	SPR3PTH = $0
127d0	012e,0000	SPR3PTL = $0

Emulation matrix

Once again, M68KWB is not meant to be an emulator. Some basic emulation is provided to check if your m68k code is behaving properly. I only implemented the features I needed, so you might be surprised some basic things are missing. A lot of them would be very easy to implement actually, so don't hesitate to contact me if you need something.

AMIGA

CPU instructions	Yes
Cycles	No cycles emulated (CPU, DMA)
Bitplanes	• Low resolution only (tested on 320x256 and 320x180 resolutions, should work for any height) • 7 bitplanes OCS HAM trick is supported (and classic HAM too) • Dual Playfield supported • EHB not yet supported
Sprites	Not yet
Sound	No
Copper	• MOVE & WAIT are ok • SKIP not implemented yet (super easy to implement though) • Despite the fact no cycles are emulated, the copper emulation still takes into account how many "pixels" it takes to execute a copper instruction depending on the number of bitplanes.
Blitter	• Blitter operations are 100% executed whenever BLTSIZE is written (kind of forced HOG). • Not all features tested yet, so far so good. The implementation was stolen from Ruper Hausberger's SAE, and butchered to fit M68KWB.
RAM	All RAM is considered as CHIP
VBL interrupt	Yes, through $6c. Reading VPOSR/VHPOSR will not work.
HBL interrupt	Nope, try using the Copper
Disk	No disk emulation, use `incbin` in your asm files if you need to load a file.

ATARI ST/STe

CPU instructions	Yes
CPU cycles	No cycles emulated (CPU, DMA)
Bitplanes	320x200, 16 colors only
Sound	Only STe digitized sound is (partially) available
HW scrolling	STe HW scrolling registers are supported (pix offset, line offset, low 8 bits of screen address)
Blitter	Yes. Note that the Blitter always behaves like in HOG mode. In other terms, whatever the HOG flag's value, the Blitter will finish its task before giving the bus back to the CPU.
VBL interrupt	Yes, through $70.w
HBL interrupt	Yes, through Timer B, in scan lines count mode. Remember though, this is not a computer emulator, and DMA cycles are not emulated. This means M68KWB assumes your HBL callback has time to execute during the HBL, and will fully execute it before drawing the next line.
Disk	No disk emulation, use `incbin` in your asm files if you need to load a file.

WHY SHOULD YOU USE THIS?

If you're an Amiga Coder

- This won't replace vasm: It does not generate a binary
- This won't replace Win/FS-UAE: It does not emulate the machine, especially not the timings. Some basic features (e.g. bitplanes) are emulated though.
- This won't replace some fabulous VS Code extensions such as Bartman's fantastic one

If you're an Atari Coder

- This won't replace vasm: It does not generate a binary
- This won't replace Hatari (or your favourite emulator): It does not emulate the machine, especially not the timings. Some basic features (e.g. bitplanes) are emulated though.

THEN WHY THE FUCK DOES M68KWB EXIST?

This thing was made to be the 1st step in your development process: Leverage the ease of use of JavaScript (JS) to prototype complex algorithms before you progressively convert them to asm. It also allows you to embed powerful bug detection features within your asm code. To be honest, even when I want to code something from scratch in asm, involving no JS at all, the iteration speed and debugging features of M68KWB are the best way to go.

WHY THE FUCK DO I SWEAR THAT MUCH?

I’m french.

THIS IS A WEB TOOL

You're not a web expert? Not a problem

You don't know anything about HTML, Javascript (JS), or CSS? Well, not a problem, really. What you really need to know is fuckin' 68K assembly ;) Though, having notions of JS, or C, C++, Java, etc. will make your life easier to use the full debugging capabilities of the system.

About File Paths

As usual in web projects, all file paths are relative to the folder where index.html sits. In other terms, "`M68KWorkbench/`" is the root folder. When registering your own FX in the system, you can redefine what the root folder is though, so that each of your FX can have its own root folder.

Tools needed to code and test

All you need to "run" the project is to open "index.html" on a server. My recommended configuration is:

- Install Visual Studio Code: https://code.visualstudio.com/
- Open the root folder (M68KWorkbench) in Visual studio Code
- Install a VS Code Extension that can Launch a development local Server, such as "Live Server" by Ritwick Dey (https://marketplace.visualstudio.com/items?itemName=ritwickdey.LiveServer). Use it to open "`index.html`".
- Install a VS Code Extension that supports M68K syntax to get your asm code look beautiful, such as m68k by Steven Tattersall (https://marketplace.visualstudio.com/items?itemName=steventattersall.m68k).

BROWSING THE FX LIST

When you open "`index.html`" for the first time, for example using “Live Server” (see above), you’re shown a box that allows you to pick the FX you want to run. Next time you’ll run M68KWB, it will skip that “select FX” box and directly run the latest FX you worked on. To select another FX than the one that is currently running, press `SHIFT+z`, it will bring the box back and allow you to select another FX.

Note that when you are stuck with a FX that keeps on starting (because its name is stored in your browser’s localStorage) and you want to start with the FX chooser window, you can add `?fx=reset` at the end of the URL, e.g:

http://127.0.0.1:5500/index.html**?fx=reset**

CREATING YOUR OWN FX

- A FX is made of 1 optional javascript file, and 1 or several optional asm files, or both. The easiest way is to duplicate a sample ;)

If you use a js file, it must implement a class that features at least 2 methods:

- **`FX_Init()`:** Called once to init the FX
- **`FX_Update()`:** Called every frame before the bitplanes are transferred to screen.

The below js methods are optional:

- **`FX_DrawDebug`**`(_ctx)`**:** Called every frame AFTER the bitplanes are transferred to screen. Call your Javascript debug code here (canvas context draws). The "_ctx" param is the canvas's context to draw to. The coordinates are your screen's coordinates.
- **FX_Onkey**(keyCode) : Called whenever a key is down. The param keyCode is the javascript keycode (see https://css-tricks.com/snippets/javascript/javascript-keycodes/) NOTE: Avoid calling ASM code (e.g. using "invoke68K") from FX_OnKey. The reason is that you are already in a keyboard event, and if you want to trace your asm code, other key events won't be processed (e.g. 't' to trace). Instead, set a flag in FX_OnKey and trigger your asm code from FX_Update when the flag is on.

Note you can have only asm files in your fx, the js is optional. Now you need to declare your fx in the system. For that, open file `user_fx.js` in the root folder, you'll find a JSON-like list of FX. The list of fields and their description can be found at the top of the file.

Congratulations, you're done! Now go code a demo!

USING THE DEBUGGER

KEY(s)	COMMAND	DESCRIPTION
a	Anchors	Show anchors/test card. Useful to see the display limits of the bitplanes and check if your FX is properly centered.

b	Bitplanes visibility	Opens a dialog box allowing to switch bitplanes on/off
alt + b	break update	trigger breakpoint at the beginning of every update (on/off)
c	cycles	toggle show CPU cycles per frame on/off
e	exit	exits current subroutine: executes code until RTS is reached
f	focus	scroll view to focus on current executed instruction
h	help	show help
j	js	toggle javascript calls from asm on/off (inline javascript on7off)
m	mem	show memory stats
o	step over	step over instruction while tracing
p	pause	toggle pause on/off
shift + p	paranoid mode	toggle paranoid mode on/off (default in on)
r	run	exit trace mode and continue execution
shift + r	restart	reloads the page and restarts the same FX, and restores breakpoints, pause and zoom
t	trace	step into (while tracing)

shift + t	Toolbox	Toggle Toolbox (the toolbox is for the moment the only draggable window)
alt + v	break vbl update	trigger breakpoint at the beginning of vbl update (on/off)
w	Where am I?	shows current instruction info + callstack and latest branches
y	Backwards	Time Machine go to prev instruction
x	Forward	Time Machine go to next instruction
shift + z	zap	clears default FX and breakpoint data and relods page

THE JAVASCRIPT / ASM PING-PONG

As seen just above, you have to implement a few methods. There is no specific feature to help you debug your pure JS code. Just use your browser's debugger that features breakpoints, watches, call stack, etc. The web is full of tutorials teaching you to debug JS code.

The **very important distinction to understand** is that you have 2 different kinds of JS in your code:

- The "*regular*" JS, located in your JS class code, within a javascript file
- The "*inline*" JS, located in your asm code using "`;>JS`"

The *inline* JS features syntax helpers that are not available in *regular* JS. The reason is that M68KWB adds an extra pass when assembling the asm code. For example, it will transform "`d0.uw`" into "`regs.d[0]&0xffff`".

So remember, the syntax you can use for *inline* JS using "`;>JS`" (see Advanced Debugging features?) is a simplified syntax that is not available in your *regular* JS methods. You'll find below a few hints to communicate between the *regular* JS part of your code, and you asm code:

- **Invoke a JS method of your FX from ASM**: `>JS MYFX.method();` The instance of the current FX is accessible through the global variable "MYFX". So you may access any member or

method using MYFX.whatever… Works both *inline* and *regular* JS, but obviously, in your *regular* JS, MYFX is the same as the JS "`this`" keyword.

- **Access registers from *regular* JS**: regs.d[] and regs.a[] described above still works in *regular* JS. BUT, all the `dn.x`, e.g. `d0.uw` does not work in *regular* JS. In other terms, you can only access registers as an array of unsigned 32 bits values when you are in *regular* JS. That being said, the TOOLS singleton provides helper functions (see below).
- **TOOLS**: TOOLS is a singleton that features many functions to make your regular JS / ASM communication smoother. Check file `src/misc/JS_ASM_Tools.js`. All functions are documented. Easier, open file `jsdoc/index.html` and find the "JS_ASM_Tools" section. Remember: every function is accessed through "TOOLS.function". So:
    - From Javascript, you will write for example:
      `t.frames = TOOLS.getLabelAdrs("precalc_frames");`
    - From ASM, you will write for example:

      ```
      ;>JS regs.a[0] = TOOLS.getLabelAdrs("precalc_frames");
      ```

One last pitfall: Javascript stores signed numbers, but the m68k stores 2-complement unsigned representation. So when using registers to assign javascript variables, you will always get the unsigned representation, e.g. if d0 contains -1, which is 0xffffffff and you do "`let foo = regs.d[0];`" then your foo variable won't be equal to -1, but to positive 0xffffffff. This is why you have conversion functions in `JS_ASM_Tools.js.` Here you would need to type:

"`let foo = TOOLS.toInt32(regs.d[0]);`"

Unless of course you want to interpret d0 as an unsigned number.

Note that when Javascript writes to an unsigned variable, it will do the 2 complement transformation by itself. So writing a javascript variable to regs.d[0] does not need a conversion.

CROSS PLATFORM DEVELOPMENT

Whaaaaaat? Not really, but still, there are a few tools to help you out prototype a piece of code for different machines.

- **Defines**

  You can use `TARGET_OCS`, `TARGET_ST` and `TARGET_STE`, e.g.

```
      IFD TARGET_ST
COLOR: dc.w $577
      ELSE
COLOR: dc.w $57f
      ENDC
```

- **MACHINE.GFX**

  MACHINE.GFX allows you to create, display and access bitplanes using the same API, whatever the target platform. Note that [Debug Primitives]() also allow that. The difference is that Debug Primitives use JavaScript Canva's draw functions on top of bitplanes while MACHINE.GFX actually writes the bitplanes (so it's affected by your palette and you can read the pixels values).

  - `MACHINE.openGFX({params})` will initialize the screen (and create a copperlist on Amiga). params are:
    - `double` : alloc and use double buffer (back buffer / front buffer)
    - `bplCount` : bitplanes count (ignored and forced to 4 on ST/STE)
    - `width` : width in pixels (ignored and forced to 320 on ST/STE)
    - `height` : height in pixels (ignored and forced to 200 on ST/STE)

    e.g.: `;>JS MACHINE.openGFX({double:true, bplCount:1, width:320, height:200})`

    Overwrites a0 with the pointer to the new backbuffer and a1 with the address of the currently shown buffer. Buffers are the contiguous bitplanes.

  - `MACHINE.GFX.update()` will perform screen swap (if double buffering selected) and update bitplanes pointers. Overwrites a0 with the pointer to the new backbuffer and a1 with the address of the currently shown buffer.
  - `MACHINE.GFX.plot(x,y,palette_index)` will assign the given palette index to the pixel at x,y (after clipping pass). e.g: `;>JS MACHINE.GFX.plot(d0.w, d1.w, 1)`
  - `MACHINE.GFX.cls()` will clear the screen e.g: `;>JS MACHINE.GFX.cls()`

LIMITATIONS & ANNOYING STUFF I NEED TO FIX

M68KWB is far from being perfect. There are things that need to be fixed, I will do someday, especially if more people start using it. For the time being, I need to spend some time coding demos too….

Assembler

- "`include`" and "`incbin`" directives are processed before IFD-like directives. So you can't conditionally include/incbin a file. In other terms, all "include"/"incbin" directives are processed.
- On very, very rare occasions (please let me know if you find a repro), the assembler will not be able to process lines where a label and an instruction are on the same line, e.g:

  "`update: moveq #0,d0`" ⇒ put the `moveq` on the next line.
- Macros inside of macros are not yet supported
- Do not declare labels inside a REPT/ENDR loop
- Don't omit size for `DC/DS/DCB/BLK`. Always add `.b, .w` or `.l`.
- Operators may be different from your favorite assembler (I tried to use the most common syntax):
  - Bitwise NOT: '~'
  - Bitwise AND: '&'
  - Bitwise OR: '!'
  - Bitwise XOR: '^'

Code and data sections

- There is a single section for code, all the code is assembled to the first 64Kb of RAM (see `CPU_CODE_SECTION_BYTES` in `config.js` to change this).
- There is a single section for all the data (incbin, DC.x, DS.x), and it's located right after the code section.
- So far so good? Almost… ALL code goes to the code section and ALL data goes to the data section, which means that if you declare data (e.g. dc.w) in the middle of code, the data address will not be near the code: It will be stored in the data section, with the other data. So if you are a fan of PC-relative code, you may not be happy. PC-relative addressing can be used though, but pc-relative instructions will be assembled and executed as non-pc-relative. The parser will simply ignore '(pc)' when parsing arguments. In other terms: you can type pc-relative code, but it will be assembled as non pc-relative code, the behavior of your code will remain the same though. OK, where's the catch then? Well, the opcode will be different, especially its size, so it can lead to different offsets if you use advanced optimizations such as auto-modified code, or jump tables upon pc-relative code .
- The "`section`" keyword is supported, but has no real effect, it just behaves as "`even`" : it makes sure the following data is 2 bytes aligned

CPU Limitations

- 68000 only. 68010,20,30,60,.... instructions are not supported.
- Clock cycles are somehow counted (to trigger interrupts), but not simulated at all. Don't ever use M68KWorkbench to know if your effect will run fast or slow on an actual machine
- Absolute data addressing: Everything is assembled as absolute long data. e.g: "`move.l a0,$70.w`" will be assembled and executed as "`move.l a0,$70.l`". In other words, you may write ".w" in your code, and it will assemble and execute, but the opcode in memory will use ".l"
- pc-relative addressing: will be assembled and executed as non-pc-relative. The parser simply ignores '(pc)' when parsing arguments.
- SR register is not emulated. All operations on the SR register are assembled as NOP. This means you can still use SR in your code (to keep compatibility with the actual machine), but there will be no effect in the emulator. If you trace the code where you put your "SR-using" instructions, you will see NOPs instead. This should not be a big problem as you may install/clear interrupt vectors anytime, it will not crash the emulator.
- The Condition Code Register (CCR) exists, you may read/write it, but it will have no effect, and don’t expect to read the current proper flags from it. **Don’t worry though, all CCR-based instructions (Bcc, Scc, ...) will work.** It's just that internally, every state flag has its own variable instead of being a bit in a SR variable. This avoids bit manipulation and makes the emulation faster. I could update the SR register for real, maybe I'll do it in the future if users (if one day I'm not the only user on earth) complain.
- TRAP has no effect in most cases (same as NOP). Only on Atari ST/STe, a few basic system calls are supported through TRAP: `ssbrk, physbase, logbase, getrez, setscreen, setpalette, setcolor, rawconio (only function 0xff to read keyboard),mxalloc, malloc`. See "function ST_Trap" in "src/atari/atariST.js" for more details. Invoking TRAP in any other context will have no effect (except an error message in the browser's log window) and the program will continue to run. If you're NOT coding for Atari ST/STe (e.g AMIGA), TRAP will do absolutely nothing.
- The following instructions are NOT supported (hopefully not necessary to prototype a FX): `ABCD, CHK, CMPM, LINK, NBCD, RESET, RTR, SBCD, STOP, TAS, TRAPV, UNLK`.

When shit hits the fan

- **VASM is complaining**

M68KWB's syntax is more permissive than vasm. On the ST (compiled in devpac mode), I met cases where vasm did not support multiple data or bss sections. I suggest you organize your code as such:

```
main.asm:
    section code
        include "source1.asm"
        include "source2.asm"
        ...
    section data
        include "all_incbin.asm"
        include "all_dc.asm"
        ...
    section bss
        include "all_bss.asm"
```

- **My browser is stuck**
    - Your browser crashes whenever you try to open its developer tools, or to trace the javascript code ⇒ Open M68KWB in a private/incognito window (just open a private window and paste the URL).
    - You will probably get more info/context opening your browser's debugger. Maybe the code execution is stuck somewhere? Maybe the console window is filled with error messages?
    - Try the good old "have you tried turning it off and on again?" ;) Seriously, close the tab, and retry.
    - Your fx is somehow stuck in an infinite loop, and everytime you launch your browser, that stupid system re-loads automatically your fx, which freezes everything again.

      ⇒ Try resetting the current fx to come back to the default menu using `?fx=reset`:

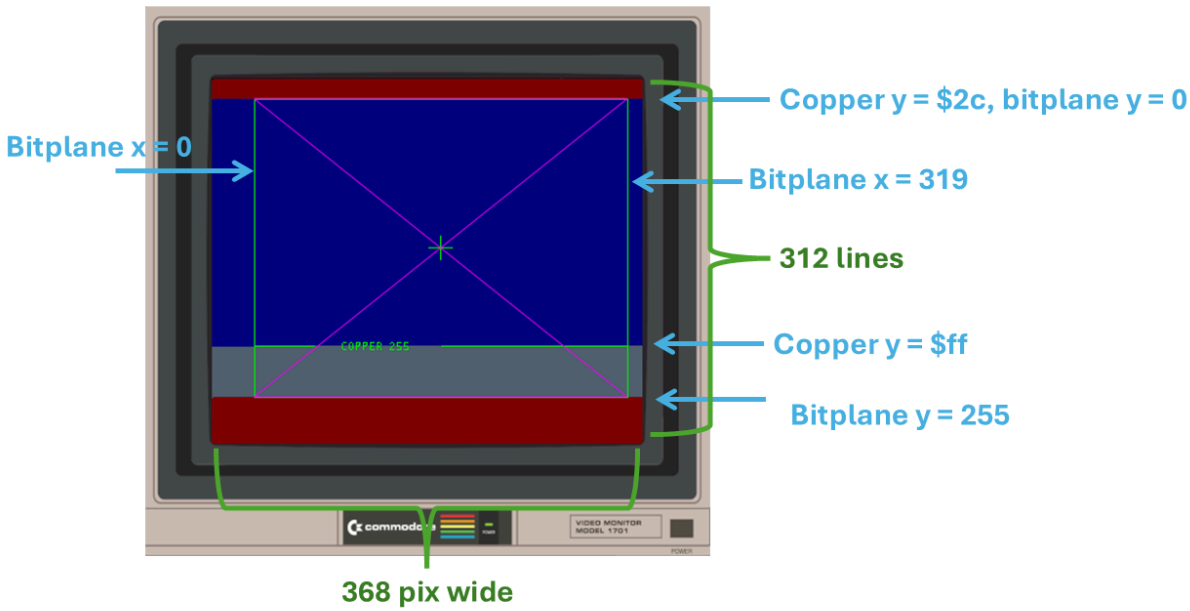
      `http://127.0.0.1:5500/index.html?fx=reset`

THE AMIGA DISPLAY

Only PAL low res is supported, hence:

- The maximum visible width is 368 pixels (http://amigadev.elowar.com/read/ADCD_2.1/Hardware_Manual_guide/node0085.html)
- The maximum visible height is 312 lines (http://amigadev.elowar.com/read/ADCD_2.1/Hardware_Manual_guide/node0063.html).

  ⇒ The monitor's image (`images/OCSMonitor.png`) is a 483x470 image that contains the monitor's frame and a 368x312 black square zone in the middle for the rendering.



The above image is generated by `fx/amiga/TestCopper`